作業：視窗登入(AI 對話)

A1143344 張鈞甯

請幫我生成一個有輸入框和按鈕的登入視窗的java程式

這是一個使用 Java Swing 建立的簡單登入視窗範例程式碼：

Java

```java
import javax.swing.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class LoginWindow {
    public static void main(String[] args) {
        // 建立主視窗 (JFrame)
        JFrame frame = new JFrame("使用者登入");
        frame.setSize(350, 200);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setLocationRelativeTo(null); // 讓視窗顯示在螢幕中央

        // 建立面板 (JPanel) 來放置元件
        JPanel panel = new JPanel();
        frame.add(panel);
        placeComponents(panel);

        // 顯示視窗
        frame.setVisible(true);
    }

    private static void placeComponents(JPanel panel) {
        // 設定面板版面佈局為自由定位 (null layout)
        panel.setLayout(null);
```

```java
// 1. 帳號標籤與輸入框
JLabel userLabel = new JLabel("帳號:");
userLabel.setBounds(40, 20, 80, 25);
panel.add(userLabel);

JTextField userText = new JTextField(20);
userText.setBounds(100, 20, 165, 25);
panel.add(userText);

// 2. 密碼標籤與輸入框 (使用 JPasswordField 隱藏密碼內容)
JLabel passwordLabel = new JLabel("密碼:");
passwordLabel.setBounds(40, 50, 80, 25);
panel.add(passwordLabel);

JPasswordField passwordText = new JPasswordField(20);
passwordText.setBounds(100, 50, 165, 25);
panel.add(passwordText);

// 3. 登入按鈕
JButton loginButton = new JButton("登入");
loginButton.setBounds(100, 90, 80, 25);
panel.add(loginButton);
```

```java
// 4. 按鈕點擊事件監聽器
loginButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        String username = userText.getText();
        String password = new String(passwordText.getPassword());

        // 簡單驗證範例
        if ("admin".equals(username) && "1234".equals(password)) {
            JOptionPane.showMessageDialog(panel, "登入成功！", "提示", JOptionPane.INFORMATION_MESSAGE);
        } else {
            JOptionPane.showMessageDialog(panel, "帳號或密碼錯誤！", "警告", JOptionPane.ERROR_MESSAGE);
        }
    }
});
```

程式重點說明：

1. `JFrame`：主視窗容器，設定視窗大小與關閉行為。
2. `JTextField` / `JPasswordField`：分別處理一般文字輸入（帳號）與隱密文字輸入（密碼）。
3. `addActionListener`：監聽登入按鈕的點擊事件，取得輸入資料並進行邏輯判斷（範例預設帳號為 `admin`，密碼為 `1234`）。
4. `JOptionPane.showMessageDialog`：跳出彈出式訊息視窗通知使用者登入結果。

為什麼class不用implements ActionListener

在這個範例中，`LoginWindow` 類別本身不需要寫 `implements ActionListener`，是因為我們使用了**匿名類別 (Anonymous Inner Class)** 來處理事件。

這兩種寫法在 Java 中都很常見，差別如下：

1. 匿名類別（範例採用的寫法）

我們直接在 `addActionListener()` 的括號內**現場建立**一個實作了 `ActionListener` 的匿名物件：

```java
loginButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        // 點擊事件寫在這裡
    }
});
```

- **優點：**
  - **職責分離：** `LoginWindow` 本身只是負責「繪製與組裝視窗」，不需要自己去變成一個「動作監聽器」。
  - **程式碼緊湊：**事件處理邏輯直接綁定在按鈕旁邊，閱讀時很直覺。
  - **可維護性高：**如果視窗上有多個按鈕（例如：登入、取消、忘記密碼），每個按鈕可以有各自獨立的匿名類別，邏輯不會混在一起。

2. 類別直接實作（`implements ActionListener` 的寫法）

如果讓 `LoginWindow` 實作 `ActionListener`，程式碼會長這樣：

```java
public class LoginWindow implements ActionListener { // 1. 宣告實作介面
    private JTextField userText;
    private JPasswordField passwordText;
    private JButton loginButton;

    public LoginWindow() {
        // ... 組件初始化 ...
        loginButton.addActionListener(this); // 2. 將自己 (this) 傳進去
    }

    // 3. 實作介面規定的方法
    @Override
    public void actionPerformed(ActionEvent e) {
        if (e.getSource() == loginButton) {
            // 處理登入邏輯
        }
    }
}
```

- **缺點：**如果介面上有很多按鈕，所有按鈕的點擊事件都會跑到同一個 `actionPerformed` 方法裡，你必須寫一堆 `if (e.getSource() == buttonA)` 或 `if (e.getSource() == buttonB)` 來判斷是誰被點擊，程式碼容易變得雜亂。

為什麼placeComponents要另外寫一個函數而不直接在主程式裡寫呢

將 `placeComponents` 獨立成一個函式，主要有以下 4 個核心好處：

1. 單一職責原則 (Single Responsibility Principle)

`main` 方法是整個 Java 程式的**啟動進入點**。它的職責是「初始化視窗、設定視窗屬性（標題、大小、關閉行為）並顯示」。
而 `placeComponents` 的職責則是「負責畫面的排版與按鈕事件」。將兩者分開，能讓每個函式只專注做一件事。

2. 提升程式碼的可讀性與可維護性

如果把所有程式碼都塞在 `main` 裡面，主程式會變得非常長且雜亂。

- **分開後**：閱讀 `main` 時，可以一眼看出整個 GUI 的架構流程（建立視窗 → 放置元件 → 顯示視窗）。
- **維護時**：若未來要修改 UI 版面或按鈕邏輯，只需要直接進入 `placeComponents` 修改，不會影響到視窗本身的設定。

3. 模組化與程式碼重用 (Reusability)

如果這套登入介面未來需要在其他地方使用（例如：點擊「切換帳號」時重新載入介面，或是切換面板 Panel），獨立成函式後就可以隨時呼叫它，而不需要重寫一遍重複的元件配置程式碼。

4. 習慣物件導向 (OOP) 的設計思維

在實際開發大型 Java 視窗應用程式時，很少會把邏輯寫在 `main` 裡面。通常會繼承 `JFrame` 並將 UI 設定拆分到不同的方法中：

Java

```java
public class LoginFrame extends JFrame {
    public LoginFrame() {
        initWindow();     // 設定視窗
        initComponents(); // 初始化並放置元件
    }

    private void initWindow() {
        setTitle("使用者登入");
        setSize(350, 200);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }

    private void initComponents() {
        // 配置元件與監聽器...
    }
}
```

這種將「建構子 / 啟動邏輯」與「介面繪製邏輯」分開寫的習慣，能讓專案架構更清晰。